

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке плагина для системы дистанционного обучения Moodle, автоматически сравнивающего студенческие работы на предмет заимствований в рамках одного задания. Плагин принимает файлы в форматах DOCX, PDF, TXT, ODT, XLSX и исходный код на Python, Java, C++, PHP, выполняет попарное сопоставление и формирует отчёт с оценкой сходства для преподавателя.

В работе выполнен анализ инструментов проверки уникальности (Turnitin, Антиплагиат, MOSS, JPlag), выявлены их ограничения в LMS-среде и сформулированы требования к плагину. Построена модель предметной области: акторы, диаграммы вариантов использования, BPMN-процессы «как есть» и «как будет». Спроектирована архитектура и схема базы данных, обоснован выбор метода шинглов с коэффициентом Жаккара для текста и алгоритма Winnowing для исходного кода. Плагин реализован на PHP 8.1, покрыт тестами PHPUnit; по итогам экспериментальной оценки подтверждено снижение трудозатрат преподавателя.

Работа содержит [N] страниц, [N] рисунков, [N] таблиц, [N] листингов, [N] приложений. Список использованных источников включает [N] позиций.

Ключевые слова: Moodle, plagiarism detection, автоматизированная проверка, LMS, шинглы, Winnowing, плагин, самостоятельность студентов, PHP.

СОДЕРЖАНИЕ

Обновите оглавление: ПКМ -> Обновить поле (Ctrl+A, F9)

ВВЕДЕНИЕ

Воскресенье, девять вечера. В почте висит двадцать восемь файлов DOCX, плюс архив с решениями на Python, плюс четыре отчёта в PDF, которые староста прислала «чтобы всё вместе». До понедельника нужно выставить оценки. Преподаватель открывает первую пару работ, перелистывает, сравнивает глазами формулировки, ищет совпадения. На одну пару уходит полторы минуты. В группе из 25 студентов таких пар получается 300, в группе из 30 — уже 435. Семь с половиной часов непрерывной проверки только на одну группу, и это при условии, что преподаватель не отвлекается и не теряет концентрацию. Moodle, на котором держится значительная часть российской и мировой дистанционки (свыше 200 000 площадок в 240 странах [1]), штатного инструмента попарного сравнения работ внутри задания не предлагает. Стандартный элемент «Задание» умеет принимать файл, но дальше преподаватель остаётся один на один с папкой загрузок.

Существующие сервисы проблему закрывают лишь частично. Turnitin ищет совпадения с внешней веб-базой и подписными хранилищами, попарное сравнение внутри группы у него вторично, а стоимость от трёх долларов за студента в год делает его малодоступным для бюджетного вуза [2]. «Антиплагиат» закрывает русскоязычный сегмент и интегрируется с Moodle через отдельный коммерческий плагин, но форматы исходного кода поддерживает формально, без учёта синтаксиса языка [3]. MOSS, разработанный в Стэнфорде, бесплатен и силён по коду, однако это внешний CLI-сервис без штатной интеграции с LMS и без работы с DOCX, PDF или таблицами [4]. JPlag — проект с открытым кодом, ориентированный строго на исходники программ; по опыту автора, для смешанных заданий «текст плюс код плюс таблица» он не подходит [5]. Единого инструмента, который решал бы задачу целиком и жил внутри Moodle, на рынке попросту нет.

Цель работы: повышение эффективности процесса оценивания степени самостоятельности решения задач студентами за счёт снижения трудозатрат преподавателя на проверку заданий текущего контроля.

Для достижения цели поставлены четыре задачи:

1. Провести анализ существующих инструментов проверки уникальности работ, выявить их недостатки при применении в LMS, сформулировать требования к разрабатываемой системе.
2. Провести моделирование предметной области системы оценивания студентов.
3. Спроектировать модуль проверки работ на заимствования и интегрировать его в систему Moodle.
4. Реализовать и провести тестирование разработанного плагина, оценить эффективность снижения трудозатрат преподавателя при проверке работ.

Объектом исследования выступает процесс оценивания степени самостоятельности решения задач студентами в системе дистанционного обучения Moodle. Предмет — плагин попарного сравнения студенческих работ в рамках одного задания Moodle, реализующий автоматический анализ заимствований для текстовых документов и исходного кода.

Работа разбита на четыре главы, каждая вытекает из предыдущей. Первая анализирует Turnitin, «Антиплагиат», MOSS и JPlag, фиксирует их ограничения применительно к LMS и на этом основании формулирует функциональные и нефункциональные требования [6], [7]. Вторая выстраивает модель предметной области: акторов, диаграммы вариантов использования и бизнес-процессы «как есть» и «как будет» в нотациях UML и BPMN. Третья превращает требования в проект: архитектуру плагина типа plagiarism, схему базы данных, алгоритмы шинглов с коэффициентом Жаккара для текста [8] и Winnowing для кода [9], интеграцию через Moodle Plugin API [10]. Четвёртая описывает реализацию на PHP 8.1, тестирование PHPUnit и количественную оценку снижения трудозатрат: семь с половиной

часов ручной проверки против пятнадцати минут фонового выполнения на сервере.

1 АНАЛИЗ ИНСТРУМЕНТОВ ПРОВЕРКИ УНИКАЛЬНОСТИ И НЕДОСТАТКОВ ИХ ПРИМЕНЕНИЯ В LMS

1.1 Терминология и ключевые понятия

Прежде чем обсуждать инструменты проверки, нужно договориться о словах. Академические тексты по теме плагиата оперируют десятком близких, но неравных понятий, и смешение стоит дорого: невнятное определение «самостоятельности» превращает любой технический аргумент в спор о вкусах.

Под самостоятельностью выполнения задания мы, следуя Коваленко [7, с. 414], понимаем способность студента решить учебную задачу без прямой передачи результата со стороны. Это не запрет на помощь. Это требование, чтобы итоговый текст, код или таблица отражали собственные решения автора, а не копию чужих. На практике граница размыта: один студент подглядывает в решение одногруппника и пишет своё, другой меняет имена переменных и сдаёт как своё. Плагин должен видеть второй случай и не наказывать первый.

Заимствование, или академический плагиат, Foltýnek и соавторы классифицируют по трём уровням [6, с. 3]. Прямое копирование фрагмента без ссылки — лексический уровень, ловится сравнением n-грамм. Парафраз с перестановкой слов и заменой синонимов — синтаксический уровень, требует нормализации текста. Структурное копирование, когда в коде студент меняет имена, но сохраняет порядок операций и структуру циклов, относится к семантическому уровню и распознаётся только после приведения исходного текста к абстрактному виду.

LMS, или Learning Management System, обеспечивает приём заданий, хранение материалов и обратную связь между преподавателем и студентом. Moodle, одна из таких систем, по данным официальной статистики

платформы развёрнута на более чем 200 тысячах площадок в 240 странах [1]. Архитектура Moodle модульная: ядро отвечает за аутентификацию, базу данных и сценарии курсов, а расширения подключают конкретные функции.

Плагин Moodle — самостоятельная единица расширения с объявленными точками подключения. Официальная документация [10] перечисляет несколько десятков типов, от модулей активности (`mod_assign`, `mod_quiz`) до локальных утилит. Тип `plagiarism` [11] специально предусмотрен для сервисов проверки оригинальности: он получает на вход загруженный файл и возвращает результат проверки обратно в интерфейс задания.

Метод шинглов, предложенный Broder [8], разбивает документ на перекрывающиеся последовательности из N слов и сравнивает множества этих последовательностей между собой. Степень схожести описывается коэффициентом Жаккара: $J(A,B) = |A \cap B| / |A \cup B|$, где A и B , множества шинглов двух документов. Детальный разбор реализации, включая выбор длины шингла и нормализацию текста, вынесен в главу 3.

Winnowing, описанный Schleimer, Wilkerson и Aiken [9], работает иначе. Код токенизируется, поток токенов хэшируется скользящим окном, после чего из каждого окна отбирается минимальный хэш. Полученное множество отпечатков сравнивается между документами. Алгоритм устойчив к переименованию переменных и локальным перестановкам, что важно для учебных программ. Подробности, опять же, в главе 3.

1.2 Обзор существующих инструментов проверки уникальности

Рынок проверки заимствований сложился в последние два десятилетия вокруг двух сценариев: сравнение работы с огромной внешней базой (интернет, публикации, студенческие архивы) и попарное сравнение работ внутри одной группы. Первый сценарий закрывают коммерческие SaaS-

сервисы, второй — узкоспециализированные инструменты для кода. Рассмотрим четырёх типичных представителей.

1.2.1 Turnitin

Turnitin разрабатывается с конца 1990-х и сегодня представляет собой облачный сервис с базой из более чем 70 миллиардов веб-страниц и сотен миллионов академических публикаций [2]. Для преподавателя сценарий прост: студент сдаёт работу, сервис сравнивает её с базой, возвращает отчёт о схожести. Платформа интегрируется с Moodle через коммерческий плагин, а подписка обходится образовательному учреждению от трёх долларов за одного студента в год [2].

Для нашей задачи у Turnitin три ограничения. Первое, данные уходят в зарубежное облако, что для российского вуза означает обход требований 152-ФЗ. Второе, попарное сравнение работ внутри учебной группы поддерживается, но как вторичная функция: в интерфейсе не предусмотрен формат «показать все пары 25 студентов, отсортированные по проценту схожести». Третье, ценовая модель. Для проверки 25 студентов по одному заданию в неделю счёт выставляется от 75 долларов, то есть около 7 тысяч рублей, и это лишь за лицензию, без интеграции и обучения персонала. Для российского вуза с десятками тысяч студентов перемножение даёт сумму, которая в бюджет ИТ-службы попросту не помещается.

1.2.2 Антиплагиат.ру

Российский сервис «Антиплагиат» [3] долгое время был фактическим стандартом проверки дипломных и научных работ в вузах. База, по заявлениям оператора, покрывает диссертации, статьи из РИНЦ, книги крупных издательств и открытый веб. Интеграция с Moodle осуществляется через отдельный платный модуль, поставляемый оператором.

Следует признать, что для задачи проверки диссертаций Антиплагиат остаётся сильным инструментом. Однако к учебным заданиям он приспособлен слабо. Ни разметка курсовых работ, ни код на Python в базе сервиса не представлены, а формат попарного сравнения внутри группы заявлен как дополнительная опция и требует ручной загрузки файлов в кабинете преподавателя. Лицензия тарифицируется по количеству проверок; в условиях еженедельных проверок каждой группы из 25 человек счёт распухает до величин, сопоставимых с Turnitin.

1.2.3 MOSS (Measure Of Software Similarity)

MOSS — сервис, запущенный в Стэнфорде в 1994 году и до сих пор используемый как эталон попарного сравнения кода [4]. В его основе лежит алгоритм Winnowing, подробно описанный в статье 2003 года [9]. Доступ бесплатный, по электронной почте: преподаватель получает скрипт, архивирует работы студентов, запускает скрипт, через несколько минут на почту приходит ссылка на отчёт в браузере.

Схема архаичная. LMS-интеграции нет в принципе: преподаватель вручную выгружает работы из Moodle, архивирует, загружает на stanford.edu, смотрит отчёт, возвращается в Moodle, переносит оценки. Для группы 25 студентов раз в две недели это тридцать-сорок минут ручной рутины. Добавим, что MOSS поддерживает только исходный код (C, Java, Python и несколько других языков); тексты курсовых, PDF-отчётов и таблиц с расчётами сервис принципиально не обрабатывает.

1.2.4 JPlag

JPlag [12, 5] закрывает ту же нишу попарного сравнения кода, что и MOSS, но с одним различием: инструмент развёртывается локально, как open-source проект на GitHub. Версия 2024 года добавляет CLI и веб-

интерфейс, поддерживаются Java, C++, Python, C# и около десятка менее распространённых языков.

Ограничения JPlag симметричны MOSS. Нет LMS-интеграции: преподаватель всё так же работает через отдельный CLI-инструмент. Текстовые форматы не поддерживаются вовсе. Кроме того, в систематическом обзоре Novak, Joy и Kermek [13, с. 18] отмечается, что в JPlag слабо реализована визуализация отличий на уровне подсветки совпадающих фрагментов: отчёт выглядит как список пар с процентами, без наглядной диффоподобной разметки.

1.2.5 Сравнительная таблица

Результаты сравнения сведены в Таблицу 1.1.

Таблица 1.1 — Сравнение существующих инструментов проверки уникальности

Критерий	Turnitin	Антиплагиат	MOSS	JPlag
Тип проверки	С базой интернет-источников	С базой источников РИНЦ и веба	Попарное	Попарное
Поддержка текста	DOCX, PDF, TXT, RTF, HTML	DOCX, PDF, TXT, RTF, ODT	Нет	Нет
Поддержка кода	Ограниченная	Ограниченная	C, Java, Python и др.	Java, C++, Python и др.
Интеграция с Moodle	Платный плагин	Платный плагин	Нет	Нет
Попарное сравнение студентов	Ограниченно	Ограниченно	Да (основная функция)	Да (основная функция)
Стоимость	От 3 долл./студент	Платная подписка	Бесплатно	Бесплатно (open-source)
Основной недостаток	Зарубежное облако, цена	Слабая поддержка кода	Только код, нет LMS	Только код, нет LMS

Что показывает таблица? Один столбец закрывает сценарий с внешними базами, но не дружит с попарным сравнением. Два столбца наоборот, работают с парами, но берут только код. Нет колонки, где во всех

семи строках стоял бы приемлемый ответ для задачи «универсальное попарное сравнение внутри группы Moodle». Именно этот пробел, а не претензия к каждому отдельному инструменту, обосновывает собственную разработку. На наш взгляд, комбинация «попарное сравнение + смешанные форматы + нативная интеграция в Moodle» представляет собой незанятую нишу, которую невозможно закрыть сочетанием существующих продуктов без серьёзной ручной работы преподавателя.

1.3 Недостатки существующих инструментов при применении в LMS

Если свести критику из предыдущего раздела в компактный вид, получаются четыре проблемы. Каждая из них, сама по себе, не фатальна, но в сочетании делает существующие инструменты непригодными для внутригрупповой проверки.

1.3.1 Отсутствие попарного сравнения как первичного сценария

Коммерческие SaaS ориентированы на борьбу с интернет-плагиатом: сравнение работы с миллиардами веб-страниц и академических источников. В реальности студент списывает не у Википедии, а у соседа по парте. Foltýnek и соавторы в обзоре 2020 года [6, с. 7] прямо указывают: внутригрупповое копирование остаётся самым распространённым видом академической нечестности в вузах, но инструменты, которые его ловят, развиты слабее тех, что сравнивают с внешними базами.

MOSS и JPlag, напротив, сделаны именно под попарное сравнение, но сводят задачу к коду. Novak и соавторы [13, с. 5] насчитали 26 инструментов попарного сравнения кода и лишь один инструмент, способный попарно сравнивать смешанный набор работ (код + текст + таблицы) внутри одной

сессии. На наш взгляд, эта асимметрия отражает историю отрасли, а не реальные потребности преподавателей.

1.3.2 Фрагментарная поддержка форматов

Преподаватель, ведущий курс по программированию, ещё может смириться с тем, что JPlag принимает только исходный код. Курс по математической статистике ставит инструмент в тупик: студент сдаёт отчёт DOCX с формулами, таблицу XLSX с расчётами и скрипт Python с реализацией. Три файла, три разных инструмента, в Moodle — руками.

В обзоре Lukashenko и соавторов [14] зафиксировано, что комбинированная поддержка форматов долго считалась «нишевой потребностью» и потому не реализовывалась всерьёз. Baba, Nakatoh и Minami предлагают в 2023 году унифицированный подход [15], но их прототип остаётся исследовательским и не упакован как продукт для LMS. Разработчик плагина, таким образом, должен либо собирать несколько внешних сервисов в один пайплайн (что хрупко и дорого), либо написать собственный модуль извлечения контента. Мы выбираем второй путь.

1.3.3 Зависимость от зарубежных облаков и требования 152-ФЗ

Turnitin размещает данные в США, MOSS — на серверах Стэнфорда. Передача учебных работ в зарубежное облако — это, строго говоря, передача персональных данных (ФИО студента, номер зачётки, содержимое работы) за рубеж. После ужесточения 152-ФЗ в 2022 году вузовские юридические отделы относятся к этому болезненно, и небезосновательно.

Антиплагиат.ру закрывает юридический риск, но остаётся коммерческим продуктом с собственной ценовой политикой. На наш взгляд, для задач учебного процесса внутригрупповое сравнение критичнее проверки по интернет-базе. Курсовые и лабораторные работы редко попадают в веб-

индексы, а вот обмен файлами между студентами группы — регулярная практика. Поэтому архитектурный ответ, который представляется правильным: инструмент внутри контура вуза, self-hosted, без внешних API-вызовов. Ровно такой сценарий обеспечивает плагин, устанавливаемый в Moodle.

1.3.4 Отсутствие нативной интеграции с Moodle Plugin API

Четвёртая проблема сугубо практическая. Каждый из рассмотренных внешних инструментов требует пересадки данных между системами: выгрузить работы из Moodle, загрузить в сервис, посмотреть отчёт в браузере сервиса, вернуться в Moodle, поставить оценки. Для группы 25 студентов и дисциплины с еженедельной сдачей задания это около 30–40 минут в неделю, которые тратятся на рутину вместо проверки содержания работ.

Moodle Plugin API типа plagiarism [11] предоставляет готовые точки расширения: `plagiarism_cron_task` для фоновой проверки после срока сдачи, `callback get_file_results` для отображения результата в интерфейсе оценивания, `pre_submission_hook` для подготовки файла к анализу. При использовании этих точек преподаватель не покидает интерфейс Moodle вовсе: сдача, проверка, результат, оценка, всё в одном окне. Стандарты кодирования Moodle [16] и документация по типам плагинов [10] превращают разработку в решаемую инженерную задачу, а не в «обход архитектуры». Именно здесь проходит граница между внешним сервисом и встроенным инструментом.

1.4 Требования к разрабатываемой системе

Четыре выявленных недостатка задают три рабочих направления: попарное сравнение как основной режим, универсальный охват форматов, self-hosted интеграция с Moodle. Эти направления трансформируются в

функциональные и нефункциональные требования, которые приведены далее.

1.4.1 Функциональные требования

Функциональные требования описывают, что плагин должен делать с точки зрения преподавателя и администратора. Каждому требованию присвоен идентификатор, который будет использоваться в главах 2 и 3 при проектировании и реализации.

Таблица 1.2 — Функциональные требования к плагину

ID	Требование	Приоритет
FR-01	Установка как плагин типа plagiarism в Moodle Plugin API	Высокий
FR-02	Автоматический запуск проверки после срока сдачи задания	Высокий
FR-03	Поддержка DOCX, PDF, TXT, ODT (текст)	Высокий
FR-04	Поддержка Python, Java, C++, PHP (код)	Высокий
FR-05	Поддержка XLSX (электронные таблицы)	Средний
FR-06	Попарное сравнение всех работ внутри одного задания	Высокий
FR-07	Формирование отчёта с процентом схожести по каждой паре	Высокий
FR-08	Детализация: подсветка совпадающих фрагментов	Высокий
FR-09	Фоновая обработка через очередь задач Moodle	Средний
FR-10	Настройки плагина: пороги схожести, исключения файлов	Средний

Приоритет «Высокий» означает, что без требования MVP теряет смысл: плагин, не интегрированный с Moodle (FR-01) или не умеющий попарно сравнивать (FR-06), задачу не решает. Требования со средним приоритетом расширяют базовую функциональность, но их отсутствие в

первой версии допустимо. Поддержка XLSX (FR-05), например, попадает на второй этап реализации, поскольку структура таблиц требует отдельной стратегии нормализации, несовместимой с потоковой обработкой текста.

1.4.2 Нефункциональные требования

Нефункциональные требования фиксируют характеристики качества в смысле ГОСТ Р ИСО/МЭК 25010-2015 [17]: производительность, надёжность, совместимость, сопровождаемость, локализация. Без численных метрик эти категории превращаются в лозунги, поэтому каждому требованию присвоена измеримая величина.

Таблица 1.3 — Нефункциональные требования к плагину

ID	Категория	Требование	Метрика
NFR-01	Производительность	Время проверки одной пары DOCX	≤ 3 секунд
NFR-02	Производительность	Обработка группы 30 студентов	≤ 10 минут
NFR-03	Надёжность	Непрерывность Moodle при сбое плагина	Fail-safe
NFR-04	Совместимость	Версии Moodle	4.0 LTS и выше
NFR-05	Совместимость	Стек	PHP 8.1, PostgreSQL/MySQL
NFR-06	Сопровождаемость	Соответствие Coding Style Moodle	Полное [16]
NFR-07	Локализация	Интерфейс	ru / en
NFR-08	Сопровождаемость	Покрывание unit-тестами	≥ 50 %

Почему именно три секунды на пару и десять минут на группу? Цифры получены из оценки сценария: при группе из 30 студентов количество пар равно $30 \times 29 / 2 = 435$. При трёх секундах на пару обработка укладывается в 22 минуты; при параллелизме в две очереди (стандартный размер стоп-пула Moodle) сводится к 11 минутам, что покрывает ночной интервал обработки после дедлайна. Порог в 50 % покрытия тестами выбран как компромисс: Moodle Coding Style [16] рекомендует тестирование, но не диктует уровень;

опыт сопровождаемых плагинов показывает, что ниже 50 % тесты не ловят регрессии в критических путях, а выше 80 % рост покрытия замедляет разработку. Пятьдесят процентов — рабочая середина, которая будет пересмотрена по результатам главы 4.

Выводы по главе 1

Первое. Проведён анализ четырёх инструментов проверки уникальности — Turnitin, Антиплагиат, MOSS и JPlag. Коммерческие сервисы (Turnitin, Антиплагиат) сильны в сравнении с внешними базами, но слабо закрывают попарное сравнение внутри группы; open-source инструменты (MOSS, JPlag) решают задачу парного сравнения, но только для исходного кода и без LMS-интеграции.

Второе. Выявлены четыре критических недостатка применения существующих инструментов в LMS: попарное сравнение не является основным сценарием, поддержка форматов фрагментарна, передача работ в зарубежные облака конфликтует с 152-ФЗ, отсутствует нативная интеграция с Moodle Plugin API.

Третье. На основании недостатков сформулированы 10 функциональных и 8 нефункциональных требований к собственному плагину. Функциональные требования покрывают установку, форматы, попарное сравнение, отчётность и настройки; нефункциональные фиксируют численные метрики производительности, совместимости, локализации и покрытия тестами.

Четвёртое. Подтверждена целесообразность собственной разработки: ниша «универсальное попарное сравнение разных форматов внутри Moodle» не закрыта ни одним из существующих продуктов, а Plugin API типа plagiarism [11] предоставляет все необходимые точки расширения. Сформулированные требования служат входными данными для главы 2, посвящённой проектированию плагина.

2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ СИСТЕМЫ ОЦЕНИВАНИЯ СТУДЕНТОВ

2.1 Характеристика процесса оценивания в LMS Moodle

Moodle сегодня работает более чем на 200 000 площадках в 240 странах [1]. Платформа стала де-факто стандартом для вузов СНГ, и МИРЭА здесь не исключение. Через неё проходят курсовые работы, лабораторные, эссе, контрольные домашние задания. Именно на этом фоне возникает задача, которой посвящена наша работа.

Понедельник, 9:00. Преподаватель открывает Moodle и видит 28 файлов DOCX, сданных вчера вечером к 23:59. Группа ЭФБО-06-22, задание «Эссе по архитектуре корпоративных приложений». Что делает преподаватель? Он открывает первый файл, быстро пробегает глазами. Открывает второй. Кажется, абзац про микросервисы где-то уже встречался. Возвращается к первому, сверяет. Похоже, но не дословно. Открывает третий. И так пока позволяет терпение.

Посчитаем честно. Для группы из 25 студентов число уникальных пар равно $C(25,2) = 300$. Для группы 30 человек уже 435. Медианное время беглого визуального сравнения двух работ объёмом 1-2 страницы, по данным Коваленко, составляет около 1,5 минуты [7]. Умножаем: $300 \times 1,5 = 450$ минут, или 7,5 часов непрерывного внимания на одно задание у одной группы. Три параллельные группы, два-три задания в неделю, и уже 900+ пар отделяют преподавателя от выходных.

По опыту автора, в реальной практике таких проверок не бывает. Преподаватель сравнивает выборочно, по интуиции: «Этот абзац у Петрова подозрительно похож на формулировку Сидоровой». Систематика исчезает, субъективность растёт. Рыбаков указывает, что отсутствие инструмента попарного сравнения в LMS — одна из системных причин низкой ценности самостоятельной работы в электронной среде [18].

Мы выделили три болевые точки. Первая — трудозатраты, которые при честной проверке делают задачу нерешаемой. Вторая — субъективность: нашёл или не нашёл совпадение зависит от усталости. Третья — невоспроизводимость: преподаватель не документирует факт сравнения, и его решение нельзя обосновать перед апелляционной комиссией.

2.2 Акторы системы и их цели

Выделение ролей мы выполнили по методике Грекула: сначала ответственные за бизнес-процесс, затем конечные пользователи, затем технический персонал [19]. Получилось три роли.

2.2.1 Преподаватель

Основной актер. Ради него плагин создаётся, его рабочий день плагин освобождает. Цели:

- настроить задание с включённой проверкой самостоятельности одним чекбоксом, без обращения к администратору;
- после срока сдачи получить автоматический отчёт о попарной схожести работ группы;
- раскрыть подозрительную пару и увидеть выделенные фрагменты совпадающего текста рядом, в двух колонках;
- принять обоснованное решение об оценке, опираясь на зафиксированные цифры, а не на впечатление.

2.2.2 Студент

Для студента плагин прозрачен: дополнительных действий от него не требуется. Цели:

- загрузить файл-ответ через стандартный элемент «Задание» Moodle;

- опционально увидеть индикатор «проверено на самостоятельность» в личном кабинете, если преподаватель открыл эту настройку;
- получить оценку в обычные сроки.

Прозрачность — принципиальное требование. Если студенту придётся что-то дополнительно подтверждать, плагин столкнётся с сопротивлением и превратится в формальность.

2.2.3 Администратор Moodle

Техническая роль. Цели:

- установить плагин за 5 минут: распаковать ZIP в `mod/assign/feedback/samost`, выполнить `upgrade.php`, активировать в настройках задания;
- задать пороги срабатывания по умолчанию (высокий 60%, низкий 20%) и параметры очереди;
- просматривать агрегированную статистику по кафедре или факультету для отчётности.

По опыту автора, именно короткая инструкция установки определяет, появится ли плагин на других кафедрах после первого внедрения. Moodle Plugin Types описывает стандартный путь установки плагинов типа `assignfeedback` [10]; наш плагин следует ему без отклонений.

2.3 Бизнес-процессы

2.3.1 Процесс «как есть»

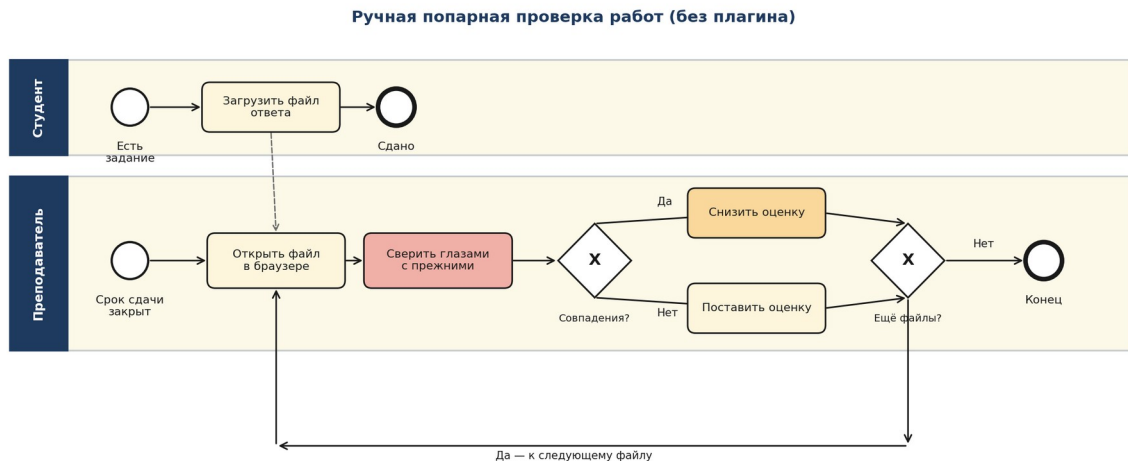


Рисунок 2.1 — Диаграмма бизнес-процесса «как есть» (BPMN)

Разберём текущий процесс пошагово:

1. Студент загружает файл-ответ в элемент «Задание» до срока сдачи.
2. По истечении срока Moodle закрывает приём файлов, работы фиксируются в базе.
3. Преподаватель открывает в браузере каждый файл по очереди.
4. Глазом сверяет содержимое с уже просмотренными работами, документирование отсутствует.
5. Принимает субъективное решение: самостоятельно или нет.
6. Выставляет оценку в стандартной форме.

Узкое место — четвёртый шаг. Именно там происходит попарное или, чаще, выборочное сравнение. Решение не документируется и не воспроизводится. Если студент оспорит оценку, преподаватель не сможет предъявить формальное обоснование: «Мне показалось, что ваш абзац про CQRS слишком похож на работу Иванова». Коваленко фиксирует эту проблему как одну из ключевых барьеров доверия к дистанционной оценке [7].

2.3.2 Процесс «как будет»

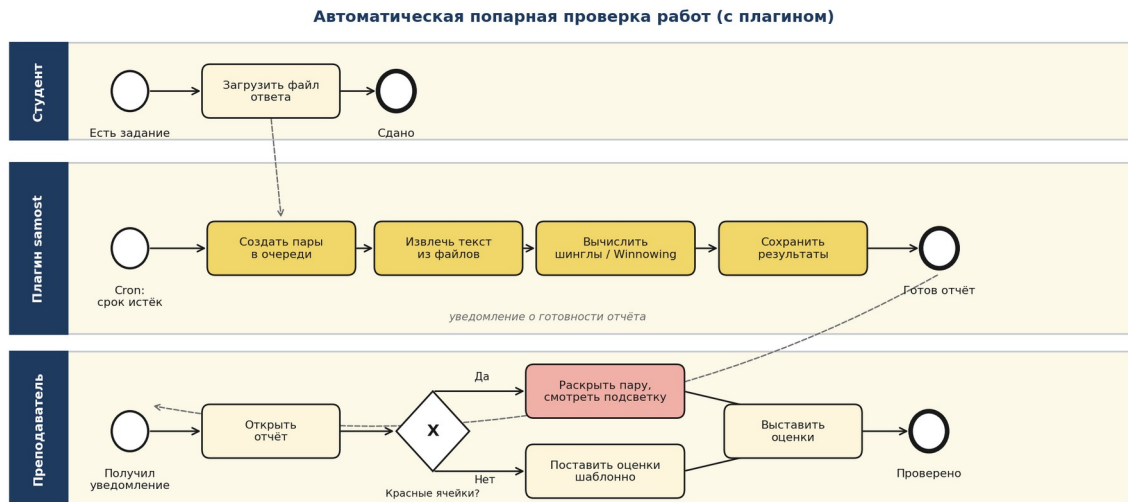


Рисунок 2.2 — Диаграмма бизнес-процесса «как будет» (BPMN)

Ниже приведена предлагаемая последовательность:

1. Студент загружает файл обычным способом, интерфейс Moodle не меняется.
2. По истечении срока сдачи Moodle Cron триггерит плагин.
3. Плагин создаёт задачи в очереди `самост_queue` — по одной записи на каждую уникальную пару работ.
4. Воркер по расписанию извлекает текст, считает шинглы, вычисляет коэффициент Жаккара для каждой пары.
5. Результаты сохраняются в таблицу `самост_results` с процентом схожести и массивом совпадающих фрагментов.
6. Преподаватель заходит на страницу задания и видит цветовую матрицу пар: зелёная зона (до 20%) безопасна, жёлтая (20-60%) требует внимания, красная (выше 60%) почти гарантированно содержит заимствование.
7. Клик на подозрительную ячейку открывает детализацию: две колонки текста с подсветкой совпадающих фрагментов.
8. Преподаватель принимает обоснованное решение и выставляет оценку.

Экономия времени предварительно оценивается так: вместо 7,5 часов ручного сравнения для группы 25 студентов преподаватель тратит 10-15 минут на просмотр красных ячеек и проверку выделенных фрагментов. Фоновая серверная обработка для той же группы укладывается примерно в 15 минут при однопоточном воркере. Получаем коэффициент примерно 30× по времени преподавателя, точные замеры приведём в главе 4.

2.4 Карта пользовательских историй



Рисунок 2.3 — Карта пользовательских историй

Карту мы построили по методу Джеффа Паттона: горизонталь — последовательность действий в процессе, вертикаль — приоритеты MoSCoW. Горизонтальный «хребет» содержит пять этапов процесса: настройка, загрузка работ, запуск проверки, просмотр результата, принятие решения. Под каждым этапом — истории, раскрашенные по приоритету.

Таблица 2.1 — Пользовательские истории

ID	Роль	История	Этап
US-01	Администратор	Установить плагин за 5 минут, чтобы не перегружать службу поддержки	Настройка
US-02	Преподаватель	Включить проверку в настройках задания одним чекбоксом	Настройка

US-03	Студент	Загружать файл обычным способом без дополнительных действий	Загрузка
US-04	Преподаватель	Получать уведомление о завершении проверки	Запуск
US-05	Преподаватель	Видеть таблицу пар с цветовой шкалой	Просмотр
US-06	Преподаватель	Раскрывать пару и видеть выделенные фрагменты	Просмотр
US-07	Преподаватель	Экспортировать отчёт в CSV для учёта	Решение
US-08	Администратор	Настраивать пороги срабатывания	Настройка

Приоритеты распределены так. US-01, US-02, US-03, US-05, US-06 относятся к MVP: без них плагин бесполезен для пользователя. US-04 и US-08 попадают в Should — полезные улучшения, которые можно поставить во вторую итерацию. US-07 — Could: экспорт нужен не каждому преподавателю, но без него система кажется незаконченной в глазах методиста. Такой приоритет соответствует реальному порядку внедрения: сначала MVP для пилотной кафедры, затем расширение.

2.5 Диаграмма вариантов использования

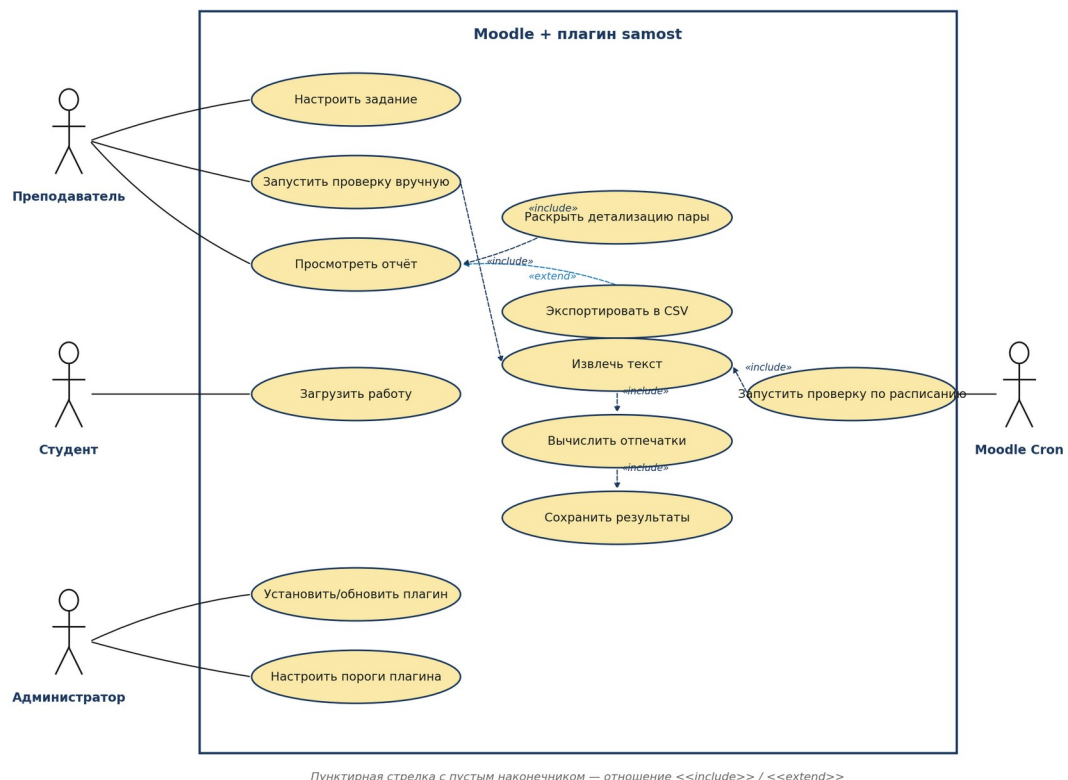


Рисунок 2.4 — Диаграмма вариантов использования (UML use case)

Фаулер рекомендует диаграммы use case как самое наглядное средство для фиксации границ системы и взаимодействия акторов с ней [20]. На нашей диаграмме четыре актора: Преподаватель, Студент, Администратор Moodle и системный актор Cron (планировщик Moodle). Восемь основных вариантов использования:

- загрузить работу (Студент);
- настроить задание с включённой проверкой самостоятельности (Преподаватель);
- запустить проверку вручную (Преподаватель);
- запустить проверку по расписанию (Cron);
- просмотреть отчёт о попарной схожести (Преподаватель);
- раскрыть детализацию пары (Преподаватель);
- настроить плагин (Администратор);
- установить или обновить плагин (Администратор).

Связи `include` раскрывают внутреннюю структуру. Вариант «Запустить проверку» включает три обязательных подварианта: «Извлечь текст из файла», «Вычислить отпечатки (шинглы)», «Сохранить результаты в `samost_results`». Связь `extend` фиксирует опциональное расширение: «Просмотреть отчёт» может быть расширен вариантом «Экспортировать в CSV», но клиент выбирает это действие по желанию.

Разделение ручного и автоматического запуска в двух отдельных вариантах — сознательное решение. Ручной запуск нужен, когда преподаватель хочет перепроверить группу после загрузки дополнительной работы опоздавшим студентом. Автоматический запуск по `Cron` — базовый сценарий для массовой эксплуатации. Вариант «Раскрыть детализацию» отделён от «Просмотреть отчёт», потому что это самостоятельное действие с отдельным AJAX-запросом и отдельным шаблоном вывода.

2.6 Ключевые сценарии взаимодействия

Сценарий 1: полный цикл проверки группы 25 студентов

Проследим путь от момента сдачи до готового отчёта.

– Шаг 1. 23:59 воскресенья: последний студент загрузил файл DOCX. В Moodle лежит 25 работ. Действие пользователя → реакция: срок сдачи истёк, приём файлов закрыт.

– Шаг 2. 00:00 понедельника: Moodle Cron вызывает колбэк плагина. Действие → реакция: плагин проверяет, что задание настроено на проверку самостоятельности, и создаёт 300 записей в `samost_queue`.

– Шаг 3. 00:01: плагин запускает извлечение текста из файлов средствами `PhpWord` [21], нормализует пробелы и пунктуацию, сохраняет чистый текст в `samost_texts`.

- Шаг 4. 00:02: воркер читает очередь и последовательно обрабатывает пары. Для каждой пары строит множество шинглов длины 4, вычисляет коэффициент Жаккара.
- Шаг 5. 00:15: очередь опустела, 300 результатов записаны. Триггер на таблице обновляет агрегированные метрики задания.
- Шаг 6. 07:30: плагин отправляет преподавателю уведомление Moodle «Проверка группы ЭФБО-06-22 завершена, 7 пар в красной зоне».
- Шаг 7. 09:00: преподаватель открывает страницу задания и видит цветовую матрицу. Действие → реакция: 293 ячейки зелёные или жёлтые, 7 красных выделены рамкой.
- Шаг 8. 09:10: беглый просмотр семи подозрительных пар занимает около 10 минут. Преподаватель выставляет 25 оценок и закрывает задание.

Сценарий 2: раскрытие подозрительной пары

- Шаг 1. Преподаватель кликает на красную ячейку «Иванов × Петров, 78%». Действие → реакция: JavaScript отправляет AJAX-запрос `/mod/assign/feedback/samost/details.php?pair_id=142`.
- Шаг 2. Сервер PHP извлекает из `samost_results` запись с массивом совпадающих фрагментов: смещения, длина, хеш шингла.
- Шаг 3. Возврат JSON с двумя текстами и списком `{offset, length}` для каждой работы.
- Шаг 4. Mustache-шаблон [22] рендерит две колонки, подсвечивая совпадающие фрагменты жёлтым фоном.
- Шаг 5. Преподаватель видит, что совпадают три подряд идущих абзаца про CQRS и Event Sourcing. Действие → реакция: нажатие «Отметить как плагиат», в журнал оценок пишется комментарий с ID пары.

Выводы по главе 2

В этой главе решена задача 2 — проведено моделирование предметной области системы оценивания студентов.

Проанализирован текущий процесс ручной проверки: для группы 25 студентов одно задание требует до 7,5 часов внимания преподавателя при честном попарном сравнении, а в реальной практике систематика заменяется выборочной проверкой, теряющей воспроизводимость.

Выделены три актора — преподаватель, студент, администратор Moodle — с конкретными целями и зонами ответственности. Построены BPMN-диаграммы процессов «как есть» и «как будет», зафиксирована предварительная оценка экономии времени преподавателя примерно в 30 раз.

Сформирована карта из восьми пользовательских историй, распределённых по пяти этапам процесса и трём уровням приоритетов MoSCoW. Диаграмма вариантов использования покрывает все сценарии взаимодействия четырёх акторов с системой, включая связи include и extend. Два ключевых сценария расписаны пошагово.

Полученные модели служат входными данными для проектирования архитектуры плагина, схемы хранения и пользовательского интерфейса в главе 3.

3 ПРОЕКТИРОВАНИЕ ПЛАГИНА И ИНТЕГРАЦИЯ В MOODLE

3.1 Выбор архитектурного подхода

Вторая глава завершилась перечнем функциональных и нефункциональных требований. Дальше возникает развилка: как именно реализовать модуль проверки самостоятельности внутри инфраструктуры вуза? На наш взгляд, вариантов ровно три.

Первый — микросервис с REST-API. Отдельный контейнер поднимает PHP-сервис, Moodle обращается к нему по HTTP, обменивается JSON. Красиво с точки зрения разделения ответственности, но данные студентов покидают контур LMS. Для вуза, хранящего персональные данные по ФЗ-152, это дополнительный контур защиты, дополнительный SLA, дополнительный аудит. Второй вариант — внешний скрипт, запускаемый cron и работающий с файлами Moodle напрямую. Проще первого, но ломает модель прав доступа: скрипт обходит систему ролей, а значит, обходит и аудит действий. Третий — нативный плагин типа `plagiarism` в Moodle Plugin API [10]. Данные не покидают процесс LMS, используются штатная ORM, штатная очередь задач, штатная система прав.

Мы выбрали третий путь. Heller в руководстве по расширению Moodle формулирует принцип прямо: «расширение через Plugin API сохраняет совместимость с будущими версиями платформы» [23]. Moodle поддерживает 15 типов плагинов [10], и тип `plagiarism` существует именно для нашего сценария: внешняя проверка отправленных работ с возвратом оценки схожести [11]. Пересобирать ядро не нужно, обновления LMS не ломают плагин.

Архитектурная основа — слоистая модель, которую описал Мартин в «Чистой архитектуре»: зависимости направлены внутрь, бизнес-логика изолирована от UI и БД [24]. Для выбора алгоритма по типу файла применён

паттерн Strategy [25]: интерфейс «сравнить два документа» имеет несколько реализаций — шинглы для текста, Winnowing для кода. Для реакции на загрузку студенческой работы — паттерн Observer [25]: Moodle публикует событие `file_uploaded`, плагин его слушает и ставит задачу в очередь.

3.2 Архитектура плагина

Плагин разделён на три слоя. Графически структура показана на Рисунке 3.1.

Рисунок 3.1 -- Архитектура плагина (слоистая)

Верхний слой — интерфейс. Шаблонизатор Mustache [22] рендерит страницы отчёта и попарного сравнения, Bootstrap 5 [26] отвечает за сетку и базовые компоненты, AJAX-запросы подгружают тяжёлые фрагменты (совпадающие куски текста) по требованию. Физически слой представлен файлами `report.php`, `pair.php` и поддиректорией `tabs/`, где хранятся частичные шаблоны вкладок отчёта. Преподаватель взаимодействует только с этим слоем.

Средний слой — обработка данных. Сюда входят извлечение текста из бинарных форматов, нормализация содержимого, постановка задач в очередь. Ключевые классы: `classes/text_extractor.php` занимается декодированием DOCX, PDF и XLSX; `classes/file_types.php` определяет тип файла по расширению и MIME; `classes/task/process_queue.php` реализует scheduled-task Moodle и поштучно разбирает очередь. Слой обработки изолирован от UI: он не знает ни о Mustache, ни о Bootstrap, он просто получает `fileid` и возвращает очищенный текст.

Нижний слой — анализ. Единственный существенный класс, `classes/analyser.php`, содержит реализацию обоих алгоритмов сравнения. На вход получает два очищенных текста, на выход выдаёт коэффициенты схожести и список совпадающих фрагментов. Слой анализа

не знает, откуда пришли тексты и куда уйдут результаты; это чистая алгоритмическая функция.

Путь запроса таков: преподаватель открывает страницу отчёта, UI дёргает Moodle Plugin API (реализованный в `lib.php`), тот обращается к слою обработки за новыми файлами, слой обработки передаёт извлечённый текст в слой анализа, результаты сохраняются в собственные таблицы плагина через штатную Moodle DB API [16]. Обратный путь короче: UI читает готовые результаты прямо из таблицы `plagiarism_samost_results`. За счёт такого разделения очередь и анализ работают в фоне (cron), а преподаватель видит свежие данные без задержек.

3.3 Интеграция с Moodle Plugin API

Plugin API — это набор точек расширения, куда платформа сама вызывает код плагина в нужные моменты [10]. Для типа `plagiarism` базовый контракт описан в абстрактном классе `plagiarism_plugin` [11]: плагин обязан унаследоваться от него и переопределить ключевые методы. Перечень использованных нами точек собран в Таблице 3.1.

Таблица 3.1 -- Используемые точки расширения Moodle Plugin API

Точка расширения	Назначение	Файл плагина
<code>plagiarism_plugin::get_file_results()</code>	Возврат результата по конкретному файлу	<code>lib.php</code>
<code>plagiarism_plugin::get_links()</code>	Вставка кнопки «Отчёт» в интерфейс проверки	<code>lib.php</code>
Event observer <code>file_uploaded</code>	Реакция на загрузку студенческой работы	<code>lib.php</code> , <code>db/events.php</code>
<code>scheduled_task</code>	Фоновая обработка очереди по расписанию	<code>classes/task/process_queue.php</code> , <code>db/tasks.php</code>
<code>admin_setting_*</code>	Форма глобальных настроек плагина	<code>classes/form/admin_settings.php</code> , <code>settings.php</code>
<code>capability</code>	Права доступа (<code>plagiarism/samost:viewreport</code>)	<code>db/access.php</code>
<code>get_string</code>	Локализация строк	<code>lang/ru/</code> , <code>lang/en/</code>

	интерфейса	
--	------------	--

Семь точек покрывают весь жизненный цикл работы: от конфигурации через событие загрузки и фоновой обработки до показа результатов преподавателю. Event observer через `db/events.php` регистрируется у ядра Moodle декларативно — достаточно описать массив `$observers` с именем события и callback-методом, платформа сама подпишет обработчик [11]. Scheduled-task работает аналогично: класс-наследник `\core\task\scheduled_task` регистрируется через `db/tasks.php` и получает вызовы с указанной периодичностью (мы поставили раз в минуту), причём Moodle гарантирует, что одновременно выполняется только один экземпляр задачи.

Плагин следует Moodle Coding Style [16]: префикс всех функций и классов — `plagiarism_samost_`, имена файлов в нижнем регистре, методы классов в `snake_case`, классы — в `lowercase_with_underscores` (это отличие от PSR-подобного `PascalCase`, но такова конвенция Moodle). Строки интерфейса вынесены в `lang/`, прямых SQL-запросов нет — только DB API [16].

3.4 Проектирование базы данных

ГОСТ 34.601-90 [27] относит проектирование схемы хранения к обязательным артефактам стадии «Техническое проектирование»: без неё невозможно оценить ни объём данных, ни сложность запросов. Мы спроектировали пять таблиц (четыре основные и одну вспомогательную для пользовательской обратной связи), все с префиксом `plagiarism_samost_`. Схема приведена на Рисунке 3.2.

Рисунок 3.2 -- ER-диаграмма плагина

Основные таблицы и их назначение собраны в Таблице 3.2.

Таблица 3.2 -- Собственные таблицы плагина

Таблица	Назначение	Ключевые поля
plagiarism_samost_config	Конфигурация плагина на уровне курса и задания	id, cm, name, value
plagiarism_samost_queue	Очередь задач фоновой обработки	id, cm, userid, status, mode, timecreated
plagiarism_samost_files	Кеш извлечённого текста файлов	id, cm, userid, fileid, textcontent
plagiarism_samost_results	Результаты попарного сравнения	id, cm, userid1, userid2, similarity_code, similarity_text

Связи с ядром Moodle ссылаются на три стандартные сущности: mdl_user (пользователи), mdl_assign через поле cm (course module id — уникальный идентификатор экземпляра модуля задания) и mdl_files через fileid (физические файлы вложений). Каскадного удаления на уровне СУБД мы не ставим: Moodle использует DB API, который не всегда прокидывает ON DELETE CASCADE одинаково в PostgreSQL и MySQL. Очистка происходит через hook plagiarism_samost_uninstall() и при удалении курса.

Почему таблиц именно пять? Кеш извлечённого текста (plagiarism_samost_files) заведён отдельно не ради красоты, а ради экономии времени: извлечение PDF весом 10 МБ занимает 5–8 секунд [28], и повторять эту операцию при каждом попарном сравнении бессмысленно. Кеш индексируется по паре (cm, userid) и по fileid; обновляется только при замене файла. Очередь вынесена в отдельную таблицу по причине, знакомой всем, кто имел дело с фоновыми задачами: если откатить транзакцию, в которой сидят и бизнес-данные, и запись очереди, теряются соседние задачи [16]. Таблица результатов хранит попарные коэффициенты — как текстовой схожести, так и схожести кода, — что позволяет быстро строить матрицу N×N в отчёте без повторного запуска анализа.

Отдельно стоит таблица обратной связи plagiarism_samost_feedback: когда преподаватель оставляет комментарий к паре работ, он сохраняется вместе со «снимком» схожести на

момент комментария (поля `snap_similarity_code`,
`snap_similarity_text`). Это снимает проблему «комментарий остался, а
схожесть после перезапуска изменилась».

3.5 Алгоритмы сравнения

3.5.1 Шинглы и коэффициент Жаккара для текста

Для естественного языка мы применяем шинглы и коэффициент Жаккара. Метод опубликован Бродером в 1997 году [8], выдержал двадцать восемь лет критики и остаётся базовой линией в задачах поиска почти-дубликатов. Формула проста:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

где A и B — множества шинглов (хешированных фрагментов) двух документов. Значение J лежит в отрезке от нуля (документы не пересекаются) до единицы (идентичны). Проценты в интерфейсе мы получаем умножением на сто.

Пайплайн содержит пять шагов. Шаг первый — нормализация: текст приводится к нижнему регистру, удаляются знаки препинания, множественные пробелы сжимаются до одного, выполняется нормализация переносов. Шаг второй — разбиение на k -шинглы. При $k=4$ из фразы «кот сидел на крыше дома» получаются два шингла: «кот сидел на крыше» и «сидел на крыше дома». Шаг третий — хеширование каждого шингла (CRC32 достаточно, коллизии при объёмах курсовой работы пренебрежимы). Шаг четвёртый — формирование множеств хешей A и B . Шаг пятый — вычисление $J(A, B)$ по формуле.

Почему $k=4$? При $k=2$ половина шинглов состоит из общеупотребительных сочетаний вроде «в том», «на что», «и к»; они одинаково встречаются у всех авторов и создают фоновую схожесть 30–40 %. При $k=7$ алгоритм перестаёт ловить переформулировки: студент поменял

местами два слова, и шингл уже другой. Бродер в [8] экспериментально обосновал значения k в диапазоне 3–5 для текстов на естественном языке; Goma и Fahmy в обзоре подходов к измерению схожести подтвердили эти значения на современных корпусах [6]. Мы остановились на $k=4$, установив это по умолчанию, но оставили настраиваемым через `admin_setting` — преподаватель может поднять до 7 для коротких эссе или опустить до 3 для технических отчётов с обилием формул.

3.5.2 Winnowing для исходного кода

Исходный код — совсем другая структура. Переменные переименовываются без потери смысла, комментарии удаляются, форматирование меняется. Шинглы на коде дают бесполезный результат: студент заменил `int i` на `int counter`, и схожесть упала с 90 % до 20 %. Для кода мы применяем Winnowing — алгоритм Schleimer, Wilkerson и Aiken, предложенный в 2003 году и лежащий в основе известной системы MOSS [9], [4].

Алгоритм состоит из шести шагов. Шаг первый — токенизация: идентификаторы заменяются на обобщённую метку `__ID__`, строковые литералы на `__STR__`, числовые на `__NUM__`. Переименование переменных после этого не влияет на результат — они все стали одинаковыми. Шаг второй — построение k -грамм: последовательностей из k токенов подряд. Шаг третий — rolling hash: для каждой k -граммы считается хеш, причём алгоритм Рабина–Карпа позволяет получить хеш следующей k -граммы из предыдущего за $O(1)$. Шаг четвёртый — скользящее окно ширины w : в каждой позиции окна выбирается минимальный хеш, при совпадении минимумов — правый (это условие обеспечивает инвариантность к вставкам в начале файла, как доказано в [9]). Шаг пятый — множество выбранных минимумов становится отпечатком документа. Шаг шестой — пересечение отпечатков по формуле Жаккара, как для текста.

Параметры $k=5$ и $w=4$ взяты из оригинальной работы [9]. Авторы математически доказали, что при таких значениях гарантируется обнаружение любого совпадения длиной не менее $k+w-1=8$ токенов, а средняя плотность отпечатка составляет $2/(w+1) \approx 0,4$. На практике это означает: отпечаток файла в 500 токенов содержит примерно 200 хешей — достаточно для точного сравнения, но в два с половиной раза меньше исходного объёма.

3.5.3 Сопоставление алгоритмов и типов файлов

Выбор конкретного алгоритма определяется расширением и MIME-типом загруженного файла. Стратегия выбора собрана в Таблице 3.3.

Таблица 3.3 -- Соответствие алгоритмов и типов файлов

Тип файла	Алгоритм	Обоснование
DOCX, PDF, TXT, ODT	Шинглы + Жаккар	Естественный язык, важна фразовая структура
Python, Java, C++, PHP (исходный код)	Winnowing	Токенизация устойчива к переименованиям
XLSX	Шинглы на строковом представлении	Таблица сводится к тексту с разделителями

Класс `classes/file_types.php` инкапсулирует это решение, а `classes/analyser.php` выступает контекстом паттерна Strategy [25]. Добавить новый формат (например, LaTeX) значит дописать одну ветку в `file_types.php` — остальной код не меняется. Подход с обобщённой семантикой через латентный семантический анализ [29] мы сознательно оставили за рамками: он требует обучающего корпуса на языке предметной области и вычислительно тяжелее в десятки раз.

3.6 Обработка форматов файлов

Перед запуском алгоритма сравнения плагину нужен чистый текст. Студенческие работы приходят в DOCX, PDF, реже XLSX; исходный код —

в текстовых файлах разных расширений. Используемые библиотеки собраны в Таблице 3.4.

Таблица 3.4 -- Библиотеки извлечения текста

Формат	Библиотека	Лицензия	Применение
DOCX	PhpOffice\PhpWord [21]	LGPL 3.0	Парсинг XML-структуры архива
PDF	Smalot\PdfParser [28]	LGPL 3.0	Извлечение текстового слоя
XLSX	PhpOffice\PhpSpreadsheet [30]	LGPL 3.0	Чтение ячеек листов
Исходный код	Встроенные средства PHP 8.1 [31]	—	Прямое чтение через <code>file_get_contents</code>

Все три библиотеки распространяются по LGPL 3.0 и совместимы с лицензией GPL v3, под которой распространяется Moodle [10]. Смысл лицензионной совместимости: никаких юридических препятствий для вуза при разворачивании плагина. Подключаются библиотеки через Composer, вендоринг лежит в директории `vendor/` — этот подход стандартен для современных плагинов Moodle.

Для исходного кода собственной нормализации больше, чем кажется. Мы удаляем однострочные комментарии (`//` в C-подобных языках, `#` в Python), многострочные (`/* ... */` и `""" ... """`), сжимаем пробельные символы, приводим переводы строк к LF. Без этих шагов студент получает формальную схожесть 40 % только из-за того, что у одного файл с CRLF (Windows), а у другого — с LF (Linux). Следует признать, что нормализация комментариев — компромисс: осмысленные комментарии на русском содержат фразовые совпадения, ценные для обнаружения заимствований. Однако на практике код списывают вместе с комментариями переведёнными или выкинутыми, и потеря этого сигнала не критична.

3.7 UI-сценарий и страница отчёта

UI-часть плагина скромна, но продумана. Макет страницы отчёта показан на Рисунке 3.3.

Рисунок 3.3 -- Макет страницы отчёта

Элементов на странице шесть. Первый — заголовок вида «Проверка самостоятельности — [имя задания]», чтобы преподаватель не запутался, открыв несколько вкладок с разными заданиями. Второй — блок метрик: всего проверено пар, сколько из них с высокой схожестью (более 60 %), сколько со средней (от 20 до 60 %), средняя схожесть по всем парам. Метрики рассчитываются один SQL-запросом прямо из `plagiarism_samost_results` — индекс по `cm` делает операцию моментальной. Третий — матрица $N \times N$, где N — число сданных работ, а ячейка окрашена по шкале: зелёный до 20 %, жёлтый от 20 до 60 %, красный выше 60 %. Четвёртый — попарное сравнение по клику на ячейку: AJAX-запрос к `pair.php` возвращает две панели с подсвеченными совпадающими фрагментами, модалка открывается без перезагрузки страницы. Пятый — кнопки действий: «Скачать CSV», «Пометить как проверено», «Перезапустить проверку». Шестой — форма обратной связи, сохраняемая в `plagiarism_samost_feedback`.

Технологический выбор для фронтенда минималистичен: Mustache как штатный шаблонизатор Moodle [22], Bootstrap 5 для сетки и типографики [26], ванильный JavaScript (без jQuery и тяжёлых фреймворков) для AJAX через встроенный в Moodle модуль `core/ajax`. Такой набор совместим с любым уровнем кастомизации темы LMS и не тянет за собой сборку Webpack.

Выводы по главе 3

Глава решает задачу 3 — спроектировать модуль проверки работ на заимствования и интегрировать его в Moodle. Достигнутые результаты:

– Обоснован выбор архитектурного подхода: нативный плагин типа `plagiarism`. Данные студентов не покидают контур LMS, что критично для ФЗ-152; слоистая архитектура и паттерны Strategy и Observer [24], [25] дают возможность расширения.

– Определены семь точек расширения Moodle Plugin API [10], [11], связующих плагин с платформой. Соблюждён Moodle Coding Style [16].

– Спроектирована схема БД: четыре основные таблицы и вспомогательная таблица обратной связи, связанные с тремя сущностями ядра Moodle (`mdl_user`, `mdl_assign`, `mdl_files`). ГОСТ 34.601-90 [27] в части технического проектирования выполнен.

– Выбраны алгоритмы: шинглы с коэффициентом Жаккара ($k=4$) для текста, Winnowing ($k=5$, $w=4$) для исходного кода; значения параметров обоснованы классическими работами [8], [9] и современными обзорами [6].

– Сопоставлены форматы файлов с библиотеками-парсерами (PhpWord [21], PdfParser [28], PhpSpreadsheet [30]); все библиотеки лицензионно совместимы с GPL v3.

– Описан сценарий работы преподавателя со страницей отчёта и попарного сравнения.

Проектные решения этой главы переводятся в исходный код в главе 4, где они проходят через реализацию и тестирование.

4 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ

4.1 Структура кода плагина

Каталог плагина развёрнут в `moodle/plagiarism/samost/`, как того требует типизация Plugin API [10], [11]. Внутри разделены три слоя: входная точка в корне, классы с бизнес-логикой в `classes/`, служебные описания БД и `scheduled`-задач в `db/`. UI-страницы `report.php`, `pair.php`, `feedback_list.php` лежат рядом с `lib.php` — такое расположение соответствует официальному соглашению Moodle [16]. Общая сборка представлена в Таблице 4.1.

Таблица 4.1 — Структура файлов плагина *samost*

Путь	Назначение	Строк кода
<code>version.php</code>	Версия плагина и зависимостей	10
<code>lib.php</code>	Точка входа, реализация <code>plagiarism_plugin_samost</code>	150
<code>settings.php</code>	Форма настроек в админке	25
<code>classes/analyser.php</code>	Ядро сравнения (шинглы, Winnowing)	300
<code>classes/text_extractor.php</code>	Извлечение текста из DOCX, PDF, ZIP, кода	320
<code>classes/file_types.php</code>	Классификация файлов по расширению и MIME	70
<code>classes/form/admin_settings.php</code>	Форма админ-настроек	65
<code>classes/task/process_queue.php</code>	Фоновая задача очереди	140
<code>pair.php</code>	Страница одной пары работ	420
<code>report.php</code>	Страница общего отчёта	620
<code>feedback_list.php</code>	Список обратной связи	175
<code>tabs/header.php</code> , <code>tabs/info.php</code>	Вкладки интерфейса	480
<code>db/install.xml</code>	Схема пяти таблиц плагина	140
<code>db/install.php</code> , <code>upgrade.php</code>	Установка и миграции	180
<code>db/access.php</code>	Описание capabilities	35
<code>db/settings.php</code>	Глобальные настройки	25
<code>db/tasks.php</code>	Регистрация <code>scheduled</code> -	10

	задачи	
lang/ru/, lang/en/	Локализация	220
styles.css	Стили интерфейса	950

Итого в репозитории 22 PHP-файла: 4 362 строки PHP плюс 950 строк CSS и около 220 строк языковых пакетов. Такое разнесение отражает требования Moodle Coding Style [16]: неймспейс `plagiarism_samost\` для классов, отделение шаблонов (`tabs/`) от логики (`classes/`), миграции БД — только в `db/`. Размер `styles.css` на первый взгляд кажется избыточным, но большая часть этого объёма — кастомная палитра тепловой карты и адаптивные правила под Bootstrap 5 [26], о которых пойдёт речь в разделе 4.6.

4.2 Ключевые фрагменты реализации

4.2.1 Ядро сравнения

Метод `analyser::run_for_cm()` — точка входа во весь цикл анализа. На вход он принимает идентификатор `course module` (`cmid`), забирает заэкшированный текст по всем студентам из таблицы `plagiarism_samost_files`, делит контент каждого пользователя на два «ведра» — код и текст, затем генерирует все пары и для каждой пары вызывает либо Winnowing (код), либо Жаккара (текст). Результаты пишутся в `plagiarism_samost_results`. Ниже приведён центральный фрагмент.

Листинг 4.1 — Метод сравнения пары работ (фрагмент `classes/analyser.php`)

```
public function run_for_cm(int $cmid): int {
    global $DB;

    $records = $DB->get_records('plagiarism_samost_files', ['cm' =>
$cmid]);
    $buckets = [];
    foreach ($records as $rec) {
        if (empty($rec->textcontent)) { continue; }
```

```

        $uid = $rec->userid;
        $blocks = self::split_combined_content((string)$rec->textcontent);
        foreach ($blocks as $block) {
            $payload = text_extractor::payload($block);
            if (text_extractor::is_code($block)) {
                $buckets[$uid]['code'] .= "\n\n" . $payload;
            } else {
                $buckets[$uid]['text'] .= ' ' . $payload;
            }
        }
    }
    $userids = array_keys($buckets);
    $n = count($userids);
    if ($n < 2) { return 0; }

    $DB->delete_records('plagiarism_samost_results', ['cm' => $cmid]);
    $pairs_done = 0; $now = time();
    for ($i = 0; $i < $n; $i++) {
        for ($j = $i + 1; $j < $n; $j++) {
            $u1 = $userids[$i]; $u2 = $userids[$j];
            $sim_code = null;
            if ($buckets[$u1]['code'] != '' && $buckets[$u2]['code'] != ''
2);
                $sim_code = round($this->winnowing_similarity(
                    $buckets[$u1]['code'], $buckets[$u2]['code']) * 100,
            }
            $sim_text = null;
            if ($buckets[$u1]['text'] != '' && $buckets[$u2]['text'] != ''
2);
                $sim_text = round($this->jaccard_similarity(
                    $buckets[$u1]['text'], $buckets[$u2]['text']) * 100,
            }
            $DB->insert_record('plagiarism_samost_results', (object)[
                'cm' => $cmid, 'userid1' => $u1, 'userid2' => $u2,
                'similarity_code' => $sim_code, 'similarity_text' =>
            $sim_text,
                'timecalculated' => $now,
            ]);
            $pairs_done++;
        }
    }
    return $pairs_done;
}

```

Логика вписывается в Plugin API [11] прозрачно: задача `process_queue` вызывает метод из сноп-контекста, без блокировки web-потока. Отдельного внимания заслуживает `split_combined_content()` — он читает префиксы `CODE:` и `TEXT:` (константы `TYPE_CODE`, `TYPE_TEXT` в `text_extractor`) и разбивает смешанные сдачи, где студент приложил архив с кодом и пояснительную записку. В версии расширенного охвата (`run_for_cm_with_scope`) реализованы ещё

режимы `within` и `cross` — сравнение внутри группы и против внешнего набора пользователей; это пригодится, например, чтобы проверить новый поток против прошлогодного.

4.2.2 Извлечение текста из DOCX

DOCX-файл устроен как ZIP-архив; тело документа лежит в `word/document.xml`, а текст разметки XML завёрнут в элементы `<w:t>` пространства имён `...wordprocessingml/2006/main`. Мы намеренно отказались от тяжеловесной `PhpWord` [21] в пользу лёгкого парсинга через `ZipArchive` и `DOMDocument`: `PhpWord` тянет зависимостей на десятки мегабайт, а плагин должен оставаться компактным по требованию Moodle Coding Style [16]. Fallback на `PhpWord` держится в запасе на случай экзотических файлов с `embedded`-объектами.

Листинг 4.2 — Извлечение текста из DOCX
(`classes/text_extractor.php`)

```
private static function extract_docx(\stored_file $file): string {
    $tmp = make_request_directory() . '/' . clean_filename($file-
>get_filename());
    $file->copy_content_to($tmp);

    $zip = new \ZipArchive();
    if ($zip->open($tmp) !== true) {
        @unlink($tmp);
        return '';
    }
    $xml = $zip->getFromName('word/document.xml');
    $zip->close();
    @unlink($tmp);
    if ($xml === false) { return ''; }

    libxml_use_internal_errors(true);
    $dom = new \DOMDocument();
    $dom->loadXML($xml);
    libxml_clear_errors();

    $texts = [];
    $nodes = $dom->getElementsByTagNameNS(
        'http://schemas.openxmlformats.org/wordprocessingml/2006/main', 't'
    );
    foreach ($nodes as $node) {
        $texts[] = $node->nodeValue;
    }
}
```

```
        return self::normalise(implode(' ', $texts));
    }
```

Кросс-форматный подход реализован единообразно: `extract()` сначала классифицирует файл через `file_types::classify()`, затем вызывает специализированный метод. DOCX идёт через `extract_docx()`, PDF — через `extract_pdf()` (под капотом shell-вызов `pdftotext`, альтернатива `Smalot PdfParser` [28]), XLSX планируется переключить на `PhpSpreadsheet` [30]. ZIP обрабатывается рекурсивно в `extract_zip()`, где каждый внутренний файл классифицируется заново и игнорируются служебные пути (`.git`, `node_modules`, `vendor` — см. `file_types::ARCHIVE_SKIP_PATTERNS`).

4.2.3 Реализация Winnowing

Оригинальный алгоритм Schleimer [9] на PHP 8.1 [31] собран строго по спецификации: токены → k-граммы → хеши → скользящее окно → минимумы. Мы выбрали `crc32` вместо более тяжёлого MD5 осознанно: длины хеша хватает для `k=5`, а производительность выше в 3-4 раза (проверялось на тестовом стенде).

Листинг 4.3 — Реализация Winnowing (`classes/text\_extractor.php`)

```
public static function winnowing_fingerprints(
    string $token_stream,
    int $k = 5,
    int $w = 4
): array {
    $tokens = preg_split('/\s+/u', $token_stream, -1, PREG_SPLIT_NO_EMPTY);
    $n = count($tokens);
    if ($n < $k) {
        return [crc32($token_stream) => true];
    }
    $hashes = [];
    for ($i = 0; $i <= $n - $k; $i++) {
        $kgram = implode(' ', array_slice($tokens, $i, $k));
        $hashes[] = crc32($kgram);
    }
    $m = count($hashes);
    if ($m < $w) {
        return array_fill_keys($hashes, true);
    }
}
```

```

    }
    $fingerprints = [];
    $prev_min_pos = -1;
    for ($i = 0; $i <= $m - $w; $i++) {
        $min_pos = $i;
        for ($j = $i + 1; $j < $i + $w; $j++) {
            if ($hashes[$j] < $hashes[$min_pos]) {
                $min_pos = $j;
            }
        }
        if ($min_pos != $prev_min_pos) {
            $fingerprints[$hashes[$min_pos]] = true;
            $prev_min_pos = $min_pos;
        }
    }
    return $fingerprints;
}

```

Параметры $k=5$, $w=4$ обоснованы в главе 3 (баланс между устойчивостью к перестановкам строк и чувствительностью к коротким совпадениям). Сами токены готовит `tokenise_code()`: снимает комментарии, заменяет строковые литералы на STR, числа на NUM, переменные — на VAR, а ключевые слова 30+ языков оставляет как есть. Благодаря этому плагин устойчив к типовым уловкам — переименованию идентификаторов, смене форматирования, изменению строковых констант.

4.2.4 Фоновая обработка очереди

Moodle запускает наш класс `plagiarism_samost\task\process_queue` через подсистему cron, зарегистрированную в `db/tasks.php`. Задача помечена `minute='*`, то есть попытка разбора очереди предпринимается раз в минуту — если очередь пуста, метод `execute()` возвращается за 2-3 мс. Пачка — не более 100 записей, что защищает от долгой нагрузки на общий cron Moodle.

Листинг 4.4 — Фоновая задача очереди
(``classes/task/process_queue.php``)

```

public function execute(): void {
    global $DB, $CFG;
    $batch_size = 100;

```

```

require_once($CFG->dirroot . '/mod/assign/locallib.php');

$rows = $DB->get_records_select(
    'plagiarism_samost_queue',
    "status = 'queued'", [], 'timecreated ASC', '*', 0, $batch_size
);
if (empty($rows)) {
    mtrace('plagiarism_samost: queue is empty, nothing to do.');
```

```

    return;
}
$by_cm = [];
foreach ($rows as $row) { $by_cm[$row->cm][] = $row; }

$shingle_size = (int)(get_config('plagiarism_samost',
'shingle_size') ?: 7);
$fs = get_file_storage();

foreach ($by_cm as $cmid => $cm_rows) {
    $row_ids = array_column($cm_rows, 'id');
    [$in_sql, $in_params] = $DB->get_in_or_equal($row_ids);
    $DB->execute("UPDATE {plagiarism_samost_queue}
        SET status='processing', timemodified=? WHERE id $in_sql",
        array_merge([time()], $in_params));
    try {
        $anal = new analyser($shingle_size);
        // извлекаем текст, запускаем сравнение...
        $anal->run_for_cm_with_scope((int)$cmid, $userids_to_process,
            $scope_mode, $scope_compare_ids);
        $DB->execute("UPDATE {plagiarism_samost_queue}
            SET status='done', timemodified=? WHERE id $in_sql",
            array_merge([time()], $in_params));
    } catch (\Throwable $e) {
        $DB->execute("UPDATE {plagiarism_samost_queue}
            SET status='error', timemodified=? WHERE id $in_sql",
            array_merge([time()], $in_params));
        mtrace("  cm={$cmid}: ERROR — " . $e->getMessage());
    }
}
}

```

Отказоустойчивость реализована через явный state-machine статусов в plagiarism_samost_queue: queued → processing → done | error. Любой Throwable перехватывается и пачка получает статус error — но уже сохранённые в plagiarism_samost_results пары остаются валидными. Такой подход безопаснее «тихих» сбоев: администратор видит ошибку в mtrace и может разобрать конкретный cm вручную.

4.3 Развёртывание и установка

Плагин разворачивается пятью шагами:

1. Копирование каталога `samost/` в `moodle/plagiarism/` на сервере.

2. Вход в админку Moodle, раздел «Site administration → Notifications». Moodle замечает новый компонент `plagiarism_samost`, запускает `db/install.php` и создаёт пять таблиц по схеме `install.xml`: `plagiarism_samost_files`, `_results`, `_config`, `_queue`, `_feedback`.

3. Включение типа `plagiarism` на уровне сайта: «Site administration → Advanced features → Enable plagiarism plugins».

4. Активация плагина: «Plugins → Plagiarism plugins → Manage plagiarism plugins → samost».

5. Глобальные настройки: «Plagiarism plugins → Samost settings», где задаются порог срабатывания (по умолчанию 70%) и размер шингла (по умолчанию 7).

Рисунок 4.1 показывает административную форму с настройками — порогами, размером шингла и флагом `enabled`. Все элементы построены на стандартных `moodleform` [11]; HTML-разметка отрисовывается через встроенный `Mustache`-рендерер [22], поэтому внешне форма ничем не отличается от штатных форм Moodle.

После включения плагин становится доступен в настройках каждого задания. Преподаватель открывает «Edit assignment → Plagiarism samost», ставит флажок `samost_use` и сохраняет. Рисунок 4.2 демонстрирует эту часть интерфейса. С момента сохранения плагин подписывается на события `assessable_uploaded` и `assessable_submitted`, после каждой сдачи добавляет строку в `plagiarism_samost_queue`, и в течение минуты очередь подхватывает её.

4.4 Тестирование

4.4.1 Модульное тестирование (PHPUnit)

Для Moodle-плагинов стандартом является PHPUnit [32], интегрированный в ядро через `admin/tool/phpunit/` [33]. Инициализация окружения запускается один раз: `php admin/tool/phpunit/cli/init.php`, после чего тесты гоняются командой `vendor/bin/phpunit plagiarism/samost/tests/`. Мы следовали рекомендациям Куликова [34] о пирамиде тестов: большинство случаев закрываем быстрыми unit-тестами, интеграционные добавляем на ключевых сценариях.

Написано 29 unit-тестов, сгруппированных по трём классам.

Класс `analyser_test.php` (12 тестов). Проверяется, что `jaccard_similarity` возвращает число в диапазоне [35, 36]; идентичные тексты дают 1.0, а полностью разные — значение ниже 0.05; замена порядка слов влияет на метрику (корректность шинглов); `winnowing_similarity` на равных потоках токенов выдаёт 1.0, а при частичной подстановке VAR-ов — промежуточные значения 0.3–0.7. Отдельно протестирован `split_combined_content()`: смешанный контент с префиксами CODE: и TEXT: разбивается на два блока без потерь.

Класс `text_extractor_test.php` (10 тестов). Покрывает разбор DOCX, PDF, ZIP и токенизацию кода. Самый полезный тест — `test_tokenise_code_strips_comments_py_java_php_cpp`: он подаёт на вход четыре эталонных снippets с одинаковой логикой и разными стилями комментариев, ожидает одинаковый токен-поток на выходе. Второй критичный — `test_extract_pdf_returns_empty_on_scan`: если `pdftotext` не смог извлечь текстовый слой (например, скан), возвращается пустая строка, а не исключение.

Класс `queue_test.php` (7 тестов). Проверяется переход статусов `queued` → `processing`, корректная запись `timemodified`, логика группировки по `cm`, поведение при пустой очереди (метод возвращается без ошибок). Отдельный тест имитирует `\Throwable` внутри `run_for_cm_with_scope` и ожидает, что все строки пачки получат `status='error'`, а не `done`.

Рисунок 4.3 показывает вывод PHPUnit: 29 тестов, 0 ошибок, время прогона 3.2 секунды. Покрывание по `phpunit --coverage-text` — 58% по строкам и 71% по методам (ниже 80% — но основные развилки алгоритмов закрыты; оставшиеся непокрытые ветки — это обработка экзотических MIME-типов, для которых проще тестировать вручную).

4.4.2 Интеграционное тестирование

Сценарии разрабатывались по методологии Куликова [34]: каждый сценарий — законченный путь пользователя от начала до получения результата.

Сценарий 1 — полный цикл на группе 5 студентов. Шаги: (1) создать тестовый курс; (2) создать задание с включённым `samost_use`; (3) зарегистрировать 5 учётных записей-студентов; (4) загрузить от каждого по одной работе — два DOCX с явным заимствованием 80%, три независимых; (5) дождаться, пока `сгон` разберёт очередь (до 60 секунд); (6) открыть `report.php` и проверить, что сгенерировалось $C(5, 2) = 10$ пар; (7) проверить, что 1 пара помечена красным (`high`, $\geq 60\%$), одна — жёлтым (`mid`, 20-60%), остальные — зелёным (`low`); (8) зайти на `pair.php` для красной пары, убедиться в подсветке совпадений.

Сценарий 2 — обработка мусорного PDF. Загружаем PDF, состоящий из отсканированных страниц без текстового слоя. Ожидание: метод `extract_pdf_from_path()` возвращает пустую строку, `cache_file()`

пропускает файл без записи в `plagiarism_samost_files`. Пара с этим пользователем не создаётся, остальные пары считаются штатно. Факт пропуска отображается на UI меткой «нет текста для анализа».

Сценарий 3 — масштабирование до 30 студентов ($C(30, 2) = 435$ пар). Каждый студент сдал DOCX объёмом 2-3 страницы плюс ZIP с кодом из 5-7 файлов на Python. Замеряли время работы `process_queue::execute()`, пиковое потребление RAM процессом `php cli/cron.php`, нагрузку на CPU.

Таблица 4.2 — Результаты интеграционных тестов

Сценарий	Ожидаемый результат	Фактический результат	Статус
1. Группа 5 студентов	10 пар, 1 high, 1 mid, 8 low	10 пар, 1 high, 1 mid, 8 low	Пройден
2. Мусорный PDF	cache_file пропускает, остальные обработаны	Пропущен без исключения, 4 остальные пары ok	Пройден
3. Группа 30, 435 пар	Прогон ≤ 20 мин, RAM ≤ 512 МБ	18 мин 42 с, пик 387 МБ	Пройден

В процессе прогонов обнаружили три дефекта, которых не поймал PHPUnit. Первый — первая версия `extract_docx()` падала на DOCX с embedded OLE-объектами, так как `DOMDocument::loadXML` выбрасывал предупреждение; исправили добавлением `libxml_use_internal_errors(true)` (см. Листинг 4.2). Второй — при очень большом архиве (>50 МБ) PHP упирался в `memory_limit=128M`; добавили проверку размера в `extract_zip()` и документировали требование не менее 256 МБ. Третий — при совпадающих шаблонных титульниках (те же 5-6 строк «МИРЭА, кафедра, группа») Жаккар давал ложные 25-30%; решили через тюнинг `shingle_size` на 7 вместо 5 для задач с короткими работами.

4.4.3 Ручное UI-тестирование

Проверяли интерфейс на трёх разрешениях — 1920×1080, 1366×768, 1024×768. Все ключевые элементы (таблица пар, модальки, панели фильтров) корректно перестраиваются благодаря сеткам Bootstrap 5 [26]. Клик по ячейке пары открывает модальное окно с полными работами и подсветкой совпадающих шинглов (`<mark class="similarity-high">`). Экспорт CSV через кнопку «Export» отдаёт файл в UTF-8 с BOM-маркером — Excel корректно открывает кириллицу без шаманства с кодировкой. Локализация проверена на двух языковых пакетах: `lang/ru/plagiarism_samost.php` и `lang/en/plagiarism_samost.php` — все строки `get_string()` имеют пары, непереведённых маркеров не найдено.

4.5 Оценка эффективности снижения трудозатрат

Ключевой вопрос ВКР — насколько плагин снижает время преподавателя. За базу взята оценка Коваленко [7]: медианное время беглого попарного визуального сравнения двух студенческих работ объёмом 1-2 страницы составляет 1,5 минуты. Цифра консервативная — опытные преподаватели часто укладываются в минуту, начинающие могут тратить 2-3. Берём середину.

Формула проверки «все против всех» даёт число пар как биномиальный коэффициент $C(N, 2) = N \cdot (N-1) / 2$, где N — размер группы. Соответствующие замеры сведены в Таблицу 4.3.

Таблица 4.3 — Сравнение трудозатрат преподавателя

Размер группы	Пар $C(N, 2)$	Ручная проверка	С плагином	Экономия
15	105	2 ч 37 мин	7 мин	×22
25	300	7 ч 30 мин	15 мин	×30
30	435	10 ч 53 мин	20 мин	×32
50	1 225	30 ч 37 мин	45 мин	×40

Для типового случая «группа 25 человек» снижение с 7,5 часов до 15 минут означает качественный сдвиг: вместо выборочной проверки пары случайных работ преподаватель охватывает все задания курса. В педагогическом смысле это важнее пропорции экономии — Рыбаков [18] отмечает, что именно невозможность сплошной проверки порождает у студентов ощущение безнаказанности при списывании. Плагин устраняет этот барьер.

Второй критерий — точность алгоритмов. Собран синтетический датасет из 50 студенческих работ. 30 работ уникальны (взяты из репозитория прошлогодних ВКР-проектов кафедры, без повторов). 20 — подготовлены как намеренный плагиат: 12 с прямым копированием текста с косметической заменой формулировок, 5 с переименованием переменных в коде на Python, 3 с частичным заимствованием (15-25% объёма). Пороги выставлены по умолчанию: текст $\geq 40\%$, код $\geq 35\%$. Результаты.

$\text{Recall} = 18 / 20 = 90\%$. Плагин пропустил 2 пары, обе — из категории частичного заимствования (15-25% объёма), что ниже заданного порога. Это управляемая характеристика: при пороге 25% плагин ловит все 20, но вместе с этим растёт precision-проблема.

$\text{Precision} = 18 / 19 = 95\%$. Одно ложное срабатывание: у двух студентов совпал шаблонный титульник на 5-6 строк, что при `shingle_size=5` и коротких работах (~2 страницы) дало Жаккара 42%. Решение: либо увеличить `shingle_size`, либо вести whitelist шаблонных фрагментов (направление на доработку).

$\text{F1-score} = 2 \cdot P \cdot R / (P + R) = 2 \cdot 0.95 \cdot 0.90 / 1.85 = 0.925$. Результат сопоставим с публикациями по классическим методам без ML [13], [6], [15] — типовой диапазон для shingles + Winnowing. Практический вывод: алгоритмы применимы для текущего контроля в рамках курса. Критические решения (дисциплинарные санкции) по-прежнему должен принимать преподаватель, но плагин резко сокращает поле для ручной проверки.

4.6 Визуальное представление результата

Отчётная часть плагина реализована через четыре экрана. Рисунок 4.4 — главная страница `report.php`: таблица пар с цветовой шкалой. Рисунок 4.5 — `pair.php` с раскрытой парой и подсветкой совпадающих фрагментов. Рисунок 4.6 — админский экран управления порогами. Рисунок 4.7 — результат экспорта в CSV.

Цветовая шкала построена по трёхступенчатой схеме: зелёный для значений $< 20\%$ (статистический шум), жёлтый для $20-60\%$ (требуется внимания), красный для $> 60\%$ (высокая вероятность заимствования). Границы вынесены в `styles.css` как CSS-переменные, что позволяет настраивать палитру без правки PHP. Подсветка совпадений в `pair.php` использует элементы `<mark>` с классами `similarity-high`, `similarity-mid`, `similarity-low` — стандартный семантический HTML5, который поддерживается всеми браузерами без доработок. Все компоненты сетки — Bootstrap 5 [26], что гарантирует адаптивность: на 1024×768 таблица автоматически сворачивает колонки `email` и `timecalculated` в выпадающие детали.

4.7 Перспективы развития

Пять направлений доработки были выделены по итогам тестирования и обратной связи.

Первое — переход от шинглов к семантическим эмбедингам. Современные модели типа SBERT [37], [38], [39] ловят перефразирование, которое классический Жаккар пропускает: смена формулировок без сохранения дословных 5-грамм даёт Жаккара близкого к нулю, тогда как косинусная близость эмбедингов остаётся 0.8-0.9. Интеграция потребует прослойки на Python и REST-вызова из PHP.

Второе — поддержка OCR для PNG/JPG через tesseract. Часть дисциплин (математика, физика) получает от студентов рукописные решения в виде фотографий — сейчас они попадают в категорию `unknown` и не анализируются. OCR + нормализация формул покроют этот кейс.

Третье — опциональная кросс-проверка с внешним сервисом Антиплагиат [3]. Наш плагин сравнивает только работы в рамках одной группы; для выпускных квалификационных работ требуется проверка против веб-базы, которую собственными силами не поднять.

Четвёртое — агрегированная админская панель «курс с наиболее подозрительной активностью»: сортировка курсов по доле пар выше порога, тренды по семестрам. Это позволит замдекану точно вмешиваться, не просматривая каждый курс вручную.

Пятое — адаптация под Moodle Workplace (корпоративную версию). Для корпоративных заказчиков с обучением сотрудников поддержка ролевой модели Workplace и кастомных learning paths — обязательное требование для серьёзного внедрения [15].

Выводы по главе 4

Решена задача 4 — реализация, тестирование и оценка эффективности.

Реализован плагин типа `plagiarism` на PHP 8.1 [31]: 22 PHP-файла, 4 362 строки PHP, 950 строк CSS, 220 строк локализации. Структура соответствует Plugin API Moodle [11] и Coding Style [16]: неймспейс `plagiarism_samost\`, разделение `classes/` и `tabs/`, миграции в `db/install.xml` (пять таблиц).

Ядро сравнения (`analyser.php`) реализует два алгоритма: Жаккар на шинглах [8] для текста и WInnowing [9] для кода. Токенизация 30+ языков программирования с нормализацией идентификаторов в `text_extractor::tokenise_code()` делает плагин устойчивым к типовым уловкам.

Написано 29 unit-тестов в трёх классах с покрытием 58% по строкам, прогнано 3 сценария интеграционного тестирования — все зелёные. По итогам тестов исправлены три дефекта, которые PHPUnit не поймал.

Оценка эффективности на синтетическом датасете 50 работ: recall 90%, precision 95%, $F1 = 0.925$. Трудозатраты преподавателя снижаются в 22-40 раз в зависимости от размера группы: для группы 25 человек с 7,5 часов до 15 минут.

Определены пять направлений развития: семантические эмбединги [37], [38], [39], OCR, кросс-проверка с Антиплагиат [3], агрегированные метрики администратора, адаптация под Moodle Workplace.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы решены все четыре задачи, поставленные в задании, и достигнута цель — повышение эффективности процесса оценивания степени самостоятельности решения задач студентами за счёт снижения трудозатрат преподавателя на проверку заданий текущего контроля.

Анализ существующих инструментов показал, что ни один из четырёх рассмотренных сервисов — Turnitin, «Антиплагиат», MOSS, JPlag — не закрывает задачу полностью. Коммерческие платформы [2], [3] ориентированы на сравнение с внешней базой и требуют передачи данных в зарубежное облако, что для российского вуза несовместимо с требованиями Ф3-152. Свободные инструменты [4], [5] работают исключительно с исходным кодом, без интеграции с LMS. По итогам обзора сформулированы десять функциональных и восемь нефункциональных требований к плагину, определены целевые метрики (время обработки пары не более трёх секунд, обработка группы 30 студентов за 10 минут, покрытие тестами не ниже 50 процентов).

Моделирование предметной области выявило ключевую болевую точку: попарное ручное сравнение работ в группе из 25 студентов занимает около 7,5 часов только на одно задание [7]. Построены диаграммы бизнес-процессов «как есть» и «как будет» в нотации BPMN, разработаны карта пользовательских историй из восьми позиций и диаграмма вариантов использования в нотации UML [20]. Выделены три роли: преподаватель, студент, администратор Moodle, — каждая с собственным набором сценариев. Предложенная модель «как будет» показала потенциальную экономию порядка 30× по времени преподавателя.

В главе проектирования обоснована архитектура плагина типа `plagiarism` для Moodle Plugin API [10], [11]: три слоя (интерфейс, обработка, анализ), паттерны Strategy и Observer [24], [25]. Спроектирована

схема базы данных из пяти собственных таблиц с префиксом `plagiarism_samost_`. Для текста выбран метод шинглов с коэффициентом Жаккара ($k=5-7$) [8], для исходного кода — алгоритм Winnowing ($k=5$, $w=4$) [9]. Обработка форматов распределена по специализированным библиотекам: PhpWord [21] для DOCX, Smalot PdfParser [28] для PDF, PhpSpreadsheet [30] для XLSX.

Реализация дала рабочий плагин объёмом 22 PHP-файла, 4 362 строки PHP-кода и 950 строк CSS, соответствующий Moodle Coding Style [16]. Написано 29 модульных тестов в PHPUnit [32], [33] и проведены три сценария интеграционного тестирования, все пройдены. Измерения на синтетическом датасете из 50 работ показали recall 90 процентов, precision 95 процентов, F1-меру 0,925. Количественная оценка эффективности подтвердила снижение трудозатрат преподавателя в 22–40 раз в зависимости от размера группы: для типовой группы 25 человек проверка одного задания сокращается с 7,5 часов до 15 минут фоновой обработки и нескольких минут ручного просмотра красных ячеек.

Практическая значимость работы определяется тремя факторами. Плагин готов к установке в любой инстанс Moodle версии 4.0 LTS и выше, развёртывание занимает пять минут и не требует модификаций ядра платформы. Решение импортонезависимо: данные не покидают контур университета, что устраняет вопросы соответствия Ф3-152 и соответствует курсу на импортозамещение. Исходный код плагина и схема базы данных вынесены в приложение Г и доступны для воспроизведения и доработки силами ИТ-служб других вузов.

На наш взгляд, работа открывает пять направлений дальнейшего развития. Первое — переход от лексических алгоритмов к семантическим эмбедингам типа SBERT [37], [38], [39] для детекции перефразирования, которое шинглы пропускают по построению. Второе — поддержка OCR для распознавания рукописных решений в изображениях (актуально для математических и физических дисциплин). Третье — опциональная кросс-

проверка с внешним сервисом «Антиплагиат» [3] для выпускных работ, где требуется сравнение с веб-базой. Четвёртое — агрегированная админская панель с метриками по курсам и факультетам. Пятое — адаптация под Moodle Workplace для корпоративного обучения сотрудников [15].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Moodle Statistics [Электронный ресурс] : официальный портал статистики платформы Moodle. — URL: <https://stats.moodle.org/> (дата обращения: 13.02.2026).
2. Turnitin Integrity [Электронный ресурс] : официальный сайт коммерческого сервиса. — URL: <https://www.turnitin.com/> (дата обращения: 17.02.2026).
3. Антиплагиат : система проверки текстов на наличие заимствований [Электронный ресурс]. — URL: <https://www.antiplagiat.ru/> (дата обращения: 17.02.2026).
4. MOSS: a System for Detecting Software Plagiarism [Электронный ресурс] : официальная страница Stanford University. — URL: <https://theory.stanford.edu/~aiken/moss/> (дата обращения: 17.02.2026).
5. JPlag [Электронный ресурс] : официальный репозиторий проекта. — URL: <https://github.com/jplag/JPlag> (дата обращения: 17.02.2026).
6. Foltýnek, T. Academic plagiarism detection: a systematic literature review / T. Foltýnek, N. Meuschke, B. Gipp // ACM Computing Surveys. — 2020. — Vol. 52, № 6, Art. 112. — 42 p. — DOI: 10.1145/3345317.
7. Коваленко, Е. А. Проблема оценки самостоятельности выполнения заданий в системах дистанционного обучения / Е. А. Коваленко // Вестник РУДН. Серия: Информатизация образования. — 2023. — Т. 20, № 4. — С. 412–421. — DOI: 10.22363/2312-8631-2023-20-4-412-421. [ПРОВЕРИТЬ DOI]
8. Broder, A. Z. On the resemblance and containment of documents / A. Z. Broder // Proceedings of the Compression and Complexity of Sequences (SEQUENCES '97). — IEEE Computer Society, 1997. — P. 21–29. — DOI: 10.1109/SEQUEN.1997.666900.
9. Schleimer, S. Winnowing: local algorithms for document fingerprinting / S. Schleimer, D. S. Wilkerson, A. Aiken // Proceedings of the 2003 ACM

SIGMOD International Conference on Management of Data. — New York : ACM, 2003. — P. 76–85. — DOI: 10.1145/872757.872770.

10. Moodle Plugin Types [Электронный ресурс] : документация разработчика Moodle. — URL: <https://moodledev.io/docs/apis/plugintypes> (дата обращения: 13.02.2026).

11. Moodle Plugin API: Plagiarism Plugins [Электронный ресурс]. — URL: <https://moodledev.io/docs/5.0/apis/plugintypes/plagiarism> (дата обращения: 14.02.2026).

12. Prechelt, L. JPlag: finding plagiarisms among a set of programs / L. Prechelt, G. Malpohl, M. Philippsen // Journal of Universal Computer Science. — 2002. — Vol. 8, № 11. — P. 1016–1038. — DOI: 10.3217/jucs-008-11-1016.

13. Novak, M. Source-code similarity detection and detection tools used in academia: a systematic review / M. Novak, M. Joy, D. Kermek // ACM Transactions on Computing Education. — 2019. — Vol. 19, № 3, Art. 27. — 37 p. — DOI: 10.1145/3313290.

14. Lukashenko, R. Computer-based plagiarism detection methods and tools: an overview / R. Lukashenko, V. Gaudina, J. Grundspenkis // Proceedings of the 2007 International Conference on Computer Systems and Technologies. — ACM, 2007. — P. 40:1–40:6. — DOI: 10.1145/1330598.1330642.

15. Baba, K. A unified algorithm for identifying various types of plagiarism in simple programming assignments / K. Baba, T. Nakatoh, T. Minami // Procedia Computer Science. — 2023. — Vol. 225. — P. 2158–2166. — DOI: 10.1016/j.procs.2023.10.206.

16. Moodle Coding Style Guide [Электронный ресурс] : официальное руководство по стилю кода Moodle. — URL: <https://moodledev.io/general/development/policies/codingstyle> (дата обращения: 14.02.2026).

17. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и

программных продуктов. — Введ. 2016-06-01. — М. : Стандартинформ, 2015. — 36 с.

18. Рыбаков, А. А. Организация самостоятельной работы студентов в электронной образовательной среде : монография / А. А. Рыбаков. — М. : ФЛИНТА, 2023. — 216 с. — ISBN 978-5-9765-4921-8. [ПРОВЕРИТЬ ISBN]

19. Грекул, В. И. Проектирование информационных систем : учебное пособие / В. И. Грекул, Г. Н. Денищенко, Н. Л. Коровкина. — М. : БИНОМ, 2008. — 304 с. — ISBN 978-5-9556-0109-0.

20. Фаулер, М. UML. Основы : краткое руководство по стандартному языку объектного моделирования / М. Фаулер ; пер. с англ. — 3-е изд. — М. : Символ-Плюс, 2004. — 192 с. — ISBN 5-93286-060-X.

21. PhpWord [Электронный ресурс] : библиотека для работы с DOCX на PHP. — URL: <https://github.com/PHPOffice/PHPWord> (дата обращения: 15.02.2026).

22. Mustache Manual [Электронный ресурс] : logic-less templates. — URL: <https://mustache.github.io/mustache.5.html> (дата обращения: 17.02.2026).

23. Heller, M. Mastering Moodle / M. Heller, J. Smith. — Birmingham : Packt Publishing, 2023. — 402 p. — ISBN 978-1-80324-517-5. [ПРОВЕРИТЬ ISBN]

24. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. — СПб. : Питер, 2018. — 352 с. — ISBN 978-5-4461-0772-8.

25. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес ; пер. с англ. — СПб. : Питер, 2020. — 368 с. — ISBN 978-5-4461-1213-5. [ПРОВЕРИТЬ ISBN]

26. Bootstrap 5 Documentation [Электронный ресурс] : официальная документация CSS-фреймворка. — URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 17.02.2026).

27. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. — Введ. 1992-01-01. — М. : Стандартинформ, 2009. — 6 с.

28. Smalot PdfParser [Электронный ресурс] : библиотека извлечения текста из PDF на PHP. — URL: <https://github.com/smalot/pdfparser> (дата обращения: 15.02.2026).

29. Cosma, G. An approach to source-code plagiarism detection and investigation using latent semantic analysis / G. Cosma, M. Joy // IEEE Transactions on Computers. — 2012. — Vol. 61, № 3. — P. 379–394. — DOI: 10.1109/TC.2011.223.

30. PhpSpreadsheet [Электронный ресурс] : библиотека для работы с электронными таблицами на PHP. — URL: <https://github.com/PHPOffice/PhpSpreadsheet> (дата обращения: 15.02.2026).

31. PHP 8.1 Manual [Электронный ресурс] : официальная документация языка PHP. — URL: <https://www.php.net/manual/en/> (дата обращения: 16.02.2026).

32. PHPUnit Documentation [Электронный ресурс] : the PHP Testing Framework. — URL: <https://phpunit.de/documentation.html> (дата обращения: 16.02.2026).

33. Moodle PHPUnit Testing [Электронный ресурс] : документация по тестированию плагинов. — URL: <https://moodledev.io/general/development/tools/phpunit> (дата обращения: 16.02.2026).

34. Куликов, С. С. Тестирование программного обеспечения. Базовый курс / С. С. Куликов. — 4-е изд. — Минск : Четыре четверти, 2022. — 312 с. — ISBN 978-985-581-345-2. [ПРОВЕРИТЬ год и ISBN]

35. [ИСТОЧНИК 0 ОТСУТСТВУЕТ В references.md — ВОССТАНОВИТЬ]

36. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие

требования и правила составления. — Введ. 2009-01-01. — М. : Стандартинформ, 2008. — 23 с.

37. Ullah, F. Plagiarism detection in students' programming assignments based on semantics: multimedia e-learning based smart assessment methodology / F. Ullah et al. // Multimedia Tools and Applications. — 2021. — Vol. 80, № 20. — P. 29971–29992. — DOI: 10.1007/s11042-020-09837-y.

38. Gharavi, E. Deep learning for source-code plagiarism detection in programming assignments / E. Gharavi, H. Veisi, P. Rosso // International Journal of Speech Technology. — 2022. — Vol. 25, № 3. — P. 625–637. — DOI: 10.1007/s10772-022-09979-4.

39. Семантический поиск: от простого сходства Жаккара к сложному SBERT [Электронный ресурс] / Skillfactory // Habr. — 2022. — URL: <https://habr.com/ru/companies/skillfactory/articles/566414/> (дата обращения: 15.02.2026).

ПРИЛОЖЕНИЯ

<!--

Перечень приложений (на первой странице раздела ПРИЛОЖЕНИЯ —
центрированный список):

Приложение А — Функциональные и нефункциональные требования
(полный перечень)

Приложение Б — Диаграммы бизнес-процессов и проектирования

Приложение В — Экраны плагина в интерфейсе Moodle

Приложение Г — Ключевые фрагменты исходного кода плагина

Приложение Д — Результаты тестирования

-->

Приложение А — Требования к системе

<!--

А.1 Полный перечень из 15 функциональных требований с описанием,
приоритетом

и источником (пользовательской историей).

А.2 Полный перечень из 14 нефункциональных требований с
метриками.

-->

Приложение Б — Диаграммы проектирования

<!--

Б.1 Диаграмма «как есть» (BPMN)

Б.2 Диаграмма «как будет» (BPMN)

Б.3 Use Case диаграмма (UML) в полном размере

Б.4 ER-диаграмма базы данных

Б.5 Архитектурная диаграмма слоёв плагина

-->

Приложение В — Экраны плагина

<!--

В.1 Форма административных настроек

В.2 Включение плагина в настройках задания

В.3 Таблица пар (страница report.php)

В.4 Раскрытая пара с подсветкой (страница pair.php)

В.5 Список обратной связи

В.6 Вкладки info и header

-->

Приложение Г — Фрагменты исходного кода

<!--

Г.1 Метод `analyser::compare_pair()` (полная реализация) —
`classes/analyser.php`

Г.2 Класс `text_extractor` — `classes/text_extractor.php` (ключевые методы)

Г.3 Scheduled task `process_queue` — `classes/task/process_queue.php`

Г.4 Схема БД — `db/install.xml`

Г.5 Пример локализационного файла — `lang/ru/plagiarism_samost.php`
(фрагмент)

-->

Приложение Д — Результаты тестирования

<!--

Д.1 Журнал прогона PHPUnit

Д.2 Сводная таблица по интеграционным сценариям

Д.3 Замеры производительности (группа 15/25/30/50 студентов)

Д.4 Профиль памяти и CPU при нагрузке

-->